WEEK 13

Capacity, Sharding, and Python Integration

Measure growth, evaluate distribution, and load public data without hiding client or data-quality decisions.

13

A capacity claim needs quantities and a time horizon

Will it scale? becomes answerable only after the workload is measurable.

DEMAND

Records, object size, read and write shapes, and peak throughput.

OBJECTIVE

Acceptable latency, errors, freshness, retention, and recovery behavior.

HORIZON

Current state, plausible growth, and the measurement that triggers change.

Scale the bottleneck only after locating it

Distribution multiplies operating work, so it should not be the first reflex.

1. **Measure**

   Workload, plans, latency, throughput, storage, and growth

2. **Remove waste**

   Fix model, query, index, batching, and retention problems

3. **Scale up**

   Add CPU, memory, or storage when one node remains suitable

4. **Copy reads**

   Use cache or replicas with explicit freshness semantics

5. **Distribute**

   Partition or shard when results justifies coordination cost

Replication and sharding solve different growth problems

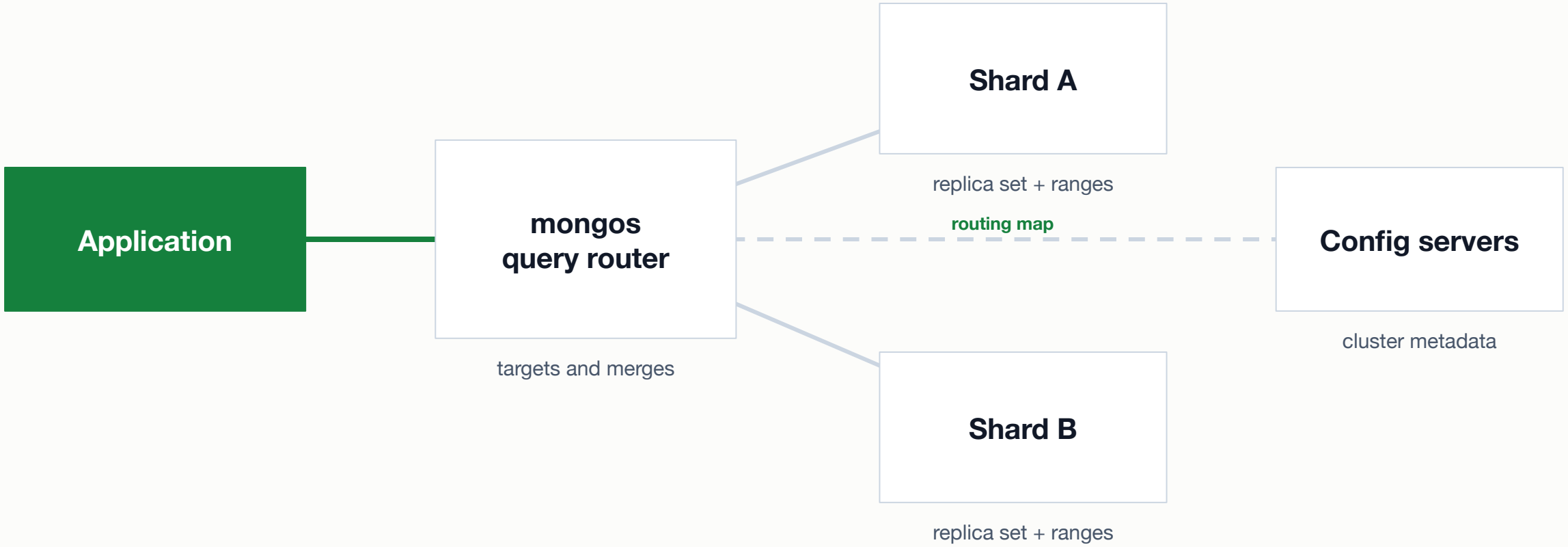
Replication

- Members maintain copies of the logical dataset
- Can support failover and selected read-routing goals
- Does not divide one collection's full storage among replicas

Sharding

- Distributes collection ranges across shards
- Can spread storage and selected read or write work
- Adds routing, metadata, balancing, and cross-shard coordination

A sharded cluster routes requests through metadata



The shard key determines distribution and whether a query can target a limited shard set.

A shard-key candidate must answer four questions

1. Cardinality: are there enough distinct values to create useful distribution choices?
2. Frequency: does one value dominate documents, requests, or bytes despite high cardinality?
3. Monotonicity: will new values continually arrive at one range extreme and create a hotspot?
4. Targeting: do important queries contain enough shard-key information to avoid broad routing?

Ranged and hashed distribution trade locality for spread

Ranged shard key

- Keeps nearby key values in nearby ranges
- Can target bounded range queries
- Skew or monotonic inserts can concentrate work

Hashed shard key

- Hashes key values to improve distribution
- Can spread monotonic identifiers
- Original-value range queries lose locality and may scatter

Candidate keys reveal different risks in the same dataset

Candidate	Cardinality/frequency	Targeting	Main risk
status	few repeated values	status filters only	hot or oversized values
dateAdded ranged	many, often monotonic	time-window queries	latest-write range hotspot
cveID hashed	high and likely spread	exact identifier lookup	time/vendor queries scatter
vendorProject + key	skew must be measured	vendor-oriented workloads	dominant vendors and complexity

Day 1 studio: Measure distribution before recommending scale

Use the notebook to audit public data and simulate candidate range and hash distribution.

- Record source, retrieval date, fields, nulls, duplicates, types, and one answerable question.
- Compare candidate cardinality, high-frequency values, monotonicity, and query targeting.
- Interpret deterministic range and hash simulations, then recommend a candidate or no sharding yet.

SUBMIT ONE THING / the first completed half of the Week 13 notebook

DAY 2

A small client should expose every important boundary

Connect safely, transform explicitly, write idempotently, and verify a real question.

The client contract differs, but safe connection habits transfer

MongoDB Atlas with PyMongo

- Runtime URI, DNS/TLS, database and collection
- Ping before writes; never normalize tlsInsecure
- Stable _id or upsert filter supports reruns

PostgreSQL/Supabase with Psycopg

- Runtime URL, TLS, role, database, schema, and table
- Use the current IPv4-compatible pooler when direct IPv6 is unreachable
- Primary key plus ON CONFLICT supports reruns

A trustworthy import has a visible before and after

A batch acknowledgment is one result; a separate read still checks stored data.

1. **Inspect**
   Source, terms, schema, size, nulls, duplicates
2. **Transform**
   Select fields and convert dates or missing values
3. **Connect**
   Enter secret at runtime, ping, and name the exact target
4. **Write**
   Use a small batch and stable key for idempotent reruns
5. **Verify**
   Compare counts, known ID, types, and grouped question

Lab: Public data capacity and cloud integration

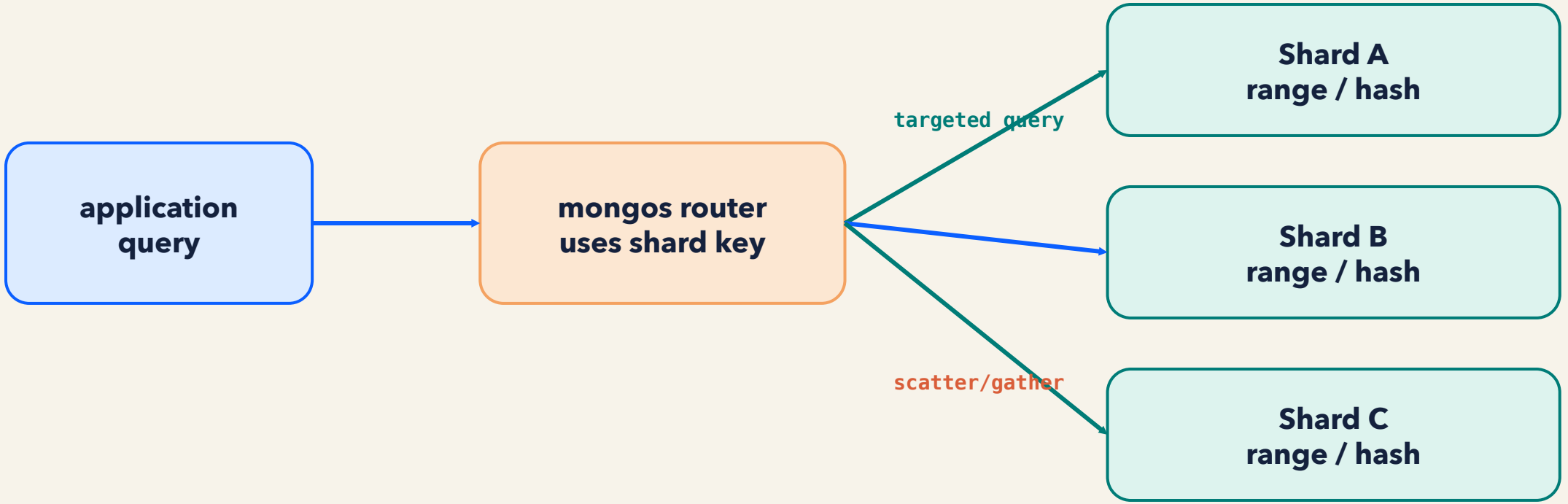
Finish one notebook that joins source audit, distribution results, a safe load, and query verification.

- Choose Atlas, PostgreSQL/Supabase, both for different responsibilities, or the full offline SQLite path.
- Load the small subset idempotently, then verify count, known identifier, types, and one grouped question.
- State one final-project connection without changing the canonical final-project requirements, and scan for secrets.

SUBMIT ONE THING / one completed 06_public_data_capacity_integration.ipynb notebook

SHARDED CLUSTER

A shard key controls routing and distribution




```
good shard key = useful cardinality + balanced frequency + query targeting + manageable growth
```

Scale decisions connect workload measurements to operating cost

Measure first, distribute only when justified, and verify every data movement.

- Which quantity would trigger a scale change?
- How do cardinality, frequency, monotonicity, and targeting interact?
- Which import results supports the claim, and which limits remain?